

CKA: Troubleshooting

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Troubleshooting Section Introduction

This article explores troubleshooting techniques for diagnosing and resolving common issues in Kubernetes clusters through hands-on labs.

Welcome to our comprehensive Kubernetes troubleshooting guide. In this article, we explore various troubleshooting techniques designed for diagnosing and resolving common issues in Kubernetes clusters. We begin by addressing application failures, then move on to troubleshooting control plane challenges, followed by worker node issues, and finally, network-related problems.

This guide is structured as a series of hands-on labs. You'll work with a simulated broken cluster to practice identifying and resolving issues. This practical approach is intended to enhance your troubleshooting skills and boost your confidence in managing production environments.

Good luck with your troubleshooting efforts and happy learning!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Application Failure

This article provides a guide on troubleshooting application failures in a two-tier architecture, focusing on systematic techniques and practical scenarios.

Welcome to this comprehensive guide on troubleshooting application failures. In this article, we review systematic troubleshooting techniques and provide practical scenarios to help you diagnose and resolve issues in a two-tier application architecture.

Overview

Consider a two-tier application consisting of a web server and a database server. The database pod hosts a database application and serves data to the web servers via a database service. Meanwhile, the web server, running on a separate pod, delivers content to the users through a web service.

Before you begin troubleshooting, document your application's configuration by creating a system map or listing all components and their connections. This documentation can be a valuable reference when determining where to start your investigation.

Troubleshooting the Web Application

When users report issues accessing your application, start by testing the application's front end. For example, if your application is web-based, verify the accessibility of the web server using a tool like curl on the service's IP address and node port:

```
```bash theme={null} curl http://web-service-ip:node-port curl: (7) Failed to connect to web-service-ip port node-port: Connection timed out
```

### ### Validate Service Endpoints

Ensure that the web service has correctly discovered endpoints for the web pod. A common issue is a mismatch between the service's endpoints and the pod's IP address.

View the service details with:

```
```bash theme={null}
bash
kubectl describe service web-service
```

You should see output similar to the following:

```
```bash theme={null} Name: web-service Namespace: default Labels: Annotations: Selector: name=webapp-mysql Type: NodePort IP: 10.96.0.156 Port: 8080/TCP TargetPort: 8080/TCP NodePort: 31672/TCP Endpoints: 10.32.0.6:8080 Session Affinity: None External Traffic Policy: Cluster Events:
```

### ### Verify Pod Configuration

Review the web application's pod definition to ensure it is correctly configured and running:

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: webapp-mysql
  labels:
    app: example-app
    name: webapp-mysql
spec:
  containers:
    - name: webapp-mysql
      image: simple-webapp-mysql
      ports:
        - containerPort: 8080
```

Next, check the pod status. A stable pod typically shows a running status with minimal restarts:

```
```bash theme={null} bash kubectl get pod
```

Example output:

```
```text theme={null}
NAME     READY   STATUS    RESTARTS   AGE
Web      1/1     Running   5          50m
```

For a more detailed inspection, including event logs, use:

```
```bash theme={null} bash kubectl describe pod web
```

This command may display events like:

```
```text theme={null}
...
Events:
Type      Reason      Age   From              Message
----      -
Normal    Scheduled   52m   default-scheduler Successfully assigned webapp-mysql to worker-1
Normal    Pulling     52m   kubelet, worker-1 pulling image "simple-webapp-mysql"
Normal    Pulled      52m   kubelet, worker-1 Successfully pulled image "simple-webapp-mysql"
Normal    Created     52m   kubelet, worker-1 Created container
Normal    Started     52m   kubelet, worker-1 Started container
```

Review Application Logs

Examine the application logs with:

```
```bash theme={null} bash kubectl logs web
```

Expected output might look like this:

```
```text theme={null}
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET /static/img/success.jpg HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET /favicon.ico HTTP/1.1" 404 -
10.32.0.1 - - [01/Apr/2019 12:51:57] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:58] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:58] "GET / HTTP/1.1" 200 -
```

If these logs do not shed light on a recent failure, continuously stream the logs to capture real-time activity:

```
```bash theme={null} bash kubectl logs web -f
```

## ## Troubleshooting the Database Service

Apply a similar approach to verify the status of the database (DB) service. Examine the following:

- \* Inspect the DB service details.
- \* Confirm that endpoints are configured correctly.
- \* Check the DB pod's status.
- \* Review the database logs for errors that might signal issues within the database.

<Callout icon="triangle-alert" color="#FF6B6B">

Ensure that both the web and DB services have matching selectors for their respective pods. Mismatches

</Callout>

## ## Visualizing the Dependency Chain

The diagram below illustrates the troubleshooting flow for dependent applications. It outlines the sequence

<Frame>

! [The image illustrates a flowchart titled "Check Dependent Applications," showing a sequence from DB to

</Frame>

## ## Additional Resources

For further troubleshooting guidance and best practices, refer to the following resources:

- \* [Kubernetes Documentation](https://kubernetes.io/docs/tasks/debug-application-cluster/troubleshoot-application/)
- \* [Kubernetes Basics](https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/)

## ## Final Tip

<Callout icon="lightbulb" color="#1CB2FE">

Use the command below to obtain detailed information about any pod. Replace `\${POD_NAME}` with the actual

</Callout>

```
```bash theme={null}
kubectl describe pods ${POD_NAME}
```

By following these steps, you can effectively troubleshoot and resolve application failures in your Kubernetes environment while ensuring a systematic approach that covers both the service and pod components.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Control Plane Failure

This guide explores methods for troubleshooting control plane failures in Kubernetes clusters, including node health checks and component status verification.

In this guide, we explore effective methods for troubleshooting control plane failures in your Kubernetes cluster. The process begins with checking the health of the nodes, followed by verifying the status of control plane components—whether they are deployed as pods or native services—and finally, reviewing detailed logs to identify any issues.

1. Verify Cluster Node Health

Begin by ensuring all cluster nodes are operational. Execute the command below to list the status of each node:

```
```bash theme={null} kubectl get nodes
```

This step helps quickly identify any node-related issues that might impact the control plane.

### ## 2. Confirm Control Plane Component Status

Control plane components can either be deployed as pods (common with kubeadm setups) or as native system services.

<Callout icon="lightbulb" color="#1CB2FE">

If your control plane components are deployed as pods in the kube-system namespace, ensure each pod is running. </Callout>

#### ### a. Checking the Kubernetes API Server

For systems using native services, inspect the API server status with:

```
```plaintext theme={null}
service kube-apiserver status
• kube-apiserver.service - Kubernetes API Server
  Loaded: loaded (/etc/systemd/system/kube-apiserver.service; enabled; vendor preset: enabled)
  Active: active (running) since Wed 2019-03-20 07:57:25 UTC; 1 weeks 1 days ago
  Docs: https://github.com/kubernetes/kubernetes
  Main PID: 15767 (kube-apiserver)
  Tasks: 13 (limit: 2362)
```

b. Checking the Controller Manager

Next, verify that the kube-controller-manager is up and running:

```
```plaintext theme={null} service kube-controller-manager status ● kube-controller-manager.service - Kubernetes
Controller Manager Loaded: loaded (/etc/systemd/system/kube-controller-manager.service; enabled; vendor
preset: enabled) Active: active (running) since Wed 2019-03-20 07:57:25 UTC; 1 weeks 1 days ago Docs:
https://github.com/kubernetes/kubernetes Main PID: 15771 (kube-controller) Tasks: 10 (limit: 2362)
```

```
c. Checking the Scheduler
```

Also, verify the status of the kube-scheduler service by running:

```
```bash theme={null}
service kube-scheduler status
```

Ensure that this command returns complete and active service details.

3. Review Control Plane Logs

After confirming the service statuses, proceed to review the logs for deeper insights.

a. Retrieving Pod Logs (When Using kubectl)

If the control plane components are deployed as pods, fetch the logs for the API server from the kube-system namespace:

```
```bash theme={null} kubectl logs kube-apiserver-master -n kube-system
```

An example excerpt from the logs might appear as follows:

```
```plaintext theme={null}
I0401 13:45:38.190735      1 server.go:703] external host was not specified, using 172.17.0.117
I0401 13:45:38.194290      1 server.go:145] Version: v1.11.3
I0401 13:45:38.819075      1 plugins.go:158] Loaded 8 mutating admission controller(s) successfully in t
NamespaceLifecycle,LimitRanger,ServiceAccount,NodeRestriction,Priority,DefaultTolerationSeconds,DefaultSto
...
W0401 13:45:41.381736      1 genericapiserver.go:319] Skipping API scheduling.k8s.io/v1alpha1 because 1
```

b. Using the Journal for Native Service Logs

For control plane components configured as native services on the master nodes, use the journal control utility to view the logs:

```
bash theme={null} sudo journalctl -u kube-apiserver
```

This command allows you to review the logs specific to the kube-apiserver service, helping in pinpointing issues.

4. Additional Resources

For further details and advanced troubleshooting techniques, refer to the official [Kubernetes Documentation](https://kubernetes.io/docs/).

By following these steps, you can systematically diagnose and address control plane issues within your Kubernetes cluster, ensuring a stable and resilient environment.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Worker Node Failure

This article explores techniques to troubleshoot worker node failures in a Kubernetes cluster, focusing on node status, kubelet health, and certificate validation.

In this article, we explore various techniques to troubleshoot worker node failures within a Kubernetes cluster. Effective troubleshooting involves checking node status, examining detailed node conditions, and diagnosing issues with the kubelet service and its certificates.

1. Check Node Status

Begin by verifying the status of the nodes in your cluster. Use the following command to determine if nodes are reporting as Ready or NotReady:

```
``plaintext theme={null} kubectl get nodes NAME STATUS ROLES AGE VERSION worker-1 Ready 8d v1.13.0 worker-2 NotReady 8d v1.13.0
```

If a node is listed as NotReady, inspect its details using:

```
``plaintext theme={null} kubectl describe node worker-1
```

This command produces an output with various conditions, such as OutOfDisk, MemoryPressure, DiskPressure, PIDPressure, and Ready. Each condition will have a status of true or false that helps pinpoint issues. For example, if disk space is insufficient, the OutOfDisk flag will be set to true; if there is low memory, the MemoryPressure flag will reflect that.

Always review the "LastHeartbeatTime" field. It indicates when a node last communicated with the master, which can provide insights if a node has unexpectedly gone down.

2. Validate Node Operation and Kubelet Health

After confirming any node issues, verify if the node itself is operational. Check the node's CPU, memory, and disk usage, review the kubelet status, inspect its logs, and ensure that the kubelet certificates are valid and correctly issued by the proper Certificate Authority (CA).

Check the Kubelet Service Status

Run the following command to check the status of the kubelet service:

```
```bash theme={null} service kubelet status
```

Example output:

```
```plaintext theme={null}
• kubelet.service - Kubernetes Kubelet
  Loaded: loaded (/etc/systemd/system/kubelet.service; enabled; vendor preset: enabled)
  Active: active (running) since Wed 2019-03-20 14:22:06 UTC; 1 weeks 1 days ago
  Docs: https://github.com/kubernetes/kubernetes
  Main PID: 1281 (kubelet)
  Tasks: 24 (limit: 1152)
```

Inspect Kubelet Logs

For further diagnosis, view the kubelet logs with:

```
```bash theme={null} sudo journalctl -u kubelet
```

An example segment of the logs may look like:

```
```plaintext theme={null}
-- Logs begin at Wed 2019-03-20 05:30:37 UTC, end at Mon 2019-04-01 14:42:42 UTC. --
Mar 20 08:12:59 worker-1 systemd[1]: Started Kubernetes Kubelet.
Mar 20 08:12:59 worker-1 kubelet[18926]: Flag --tls-cert-file has been deprecated, This parameter should be removed.
Mar 20 08:12:59 worker-1 kubelet[18926]: Flag --tls-private-key-file has been deprecated, This parameter should be removed.
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.915179 18926 flags.go:33] FLAG: --address="0.0.0.0"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918149 18926 flags.go:33] FLAG: --allow-privilege-escalation="true"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918339 18926 flags.go:33] FLAG: --allowed-unsafe-ports="[]"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918520 18926 flags.go:33] FLAG: --also-log-test-logs="false"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918621 18926 flags.go:33] FLAG: --anonymous-auth="false"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918740 18926 flags.go:33] FLAG: --application-configuration="{}"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919874 18926 flags.go:33] FLAG: --authentication-token-webhook="false"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919929 18926 flags.go:33] FLAG: --authentication-token-webhook-cache-ttl="5m"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919946 18926 flags.go:33] FLAG: --authorization-mode="Webhook,OpenID"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919948 18926 flags.go:33] FLAG: --authorization-mode="Webhook,OpenID"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919950 18926 flags.go:33] FLAG: --authorization-mode="Webhook,OpenID"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919958 18926 flags.go:33] FLAG: --authorization-mode="Webhook,OpenID"
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.920160 18926 flags.go:33] FLAG: --bootstrap-kubeconfig=""
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.920172 18926 flags.go:33] FLAG: --bootstrap-kubeconfig=""
```

3. Verify Kubelet Certificates

Ensuring that the kubelet certificates are valid and correctly issued is crucial. Use the following command to inspect a kubelet certificate:

```
```bash theme={null} openssl x509 -in /var/lib/kubelet/worker-1.crt -text
```

A valid certificate should display details such as:

```
```plaintext theme={null}
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      ff:e0:23:9d:fc:78:03:35
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: CN = KUBERNETES-CA
    Validity
      Not Before: Mar 20 08:09:29 2019 GMT
      Not After : Apr 19 08:09:29 2019 GMT
    Subject: CN = system:node:worker-1, O = system:nodes
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      ...
```

Be sure that the certificate is issued by the correct CA and that none of the certificate parameters (e.g., validity period) indicate an impending or current issue.

Conclusion

By following the steps outlined above, you can efficiently troubleshoot worker node failures in your Kubernetes cluster. Regularly monitoring node conditions, validating the health of the kubelet service, and ensuring certificate integrity will help maintain a stable and robust cluster operation.

For further learning, consider exploring additional resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)

Happy troubleshooting!